Cola de comandos do robô

EST-ROBOT, criado por Andrey Violante, para uso pedagógico · andreyviolante.github.io/est-robot · mais exemplos no simulador e no guia do professor

```
from robo import *          # sempre a primeira linha

andar(rotacoes=2)           # cada comando numa linha
virar_direita(90)
```

Para mandar para o robô: **`Ctrl+Shift+B`**. O robô começa a andar logo: deixe-o no chão.

Mover

Comando	O que faz
`andar(rotacoes=2)`	anda para frente (também: `segundos=1`, `graus=360`, `potencia=30`)
`re(rotacoes=1)`	anda de ré
`andar_cm(20)`	anda 20 centímetros
`virar_direita(90)` / `virar_esquerda(90)`	vira no lugar
`motores(30, 10)`	liga os motores (esquerdo, direito) e deixa ligados
`parar()`	para os motores
`esperar(1)`	espera 1 segundo

Sensores

Comando	Dá
`luz_refletida(1)`	luz no sensor 1: 0 (escuro) a 100 (claro)
`cor_vista(1)`	a cor: compare com `PRETO`, `AZUL`, `VERDE`, `AMARELO`, `VERMELHO`, `BRANCO`, `MARROM` (ex.: `se(cor_vista(1) == VERDE)`)
`distancia(4)`	distância em centímetros (ultrassom)
`tocado(4)`	1 se o botão de toque estiver apertado, 0 se não

Repetir e decidir

```
with repetir(4):                # repete 4 vezes
    andar_cm(30)
    virar_direita(90)

with para_sempre():             # repete sem parar
    with se_ver_cor(1, PRETO):  # se o sensor 1 vê preto...
        motores(10, 30)
    with senao():               # ...senão
        motores(30, 10)

with se(distancia(4) < 15):     # se a distância for menor que 15 cm
    parar()

with repetir_ate(lambda: distancia(4) < 15):   # repete até chegar perto
    motores(30, 30)
```

Também dá para usar o `if` e o `while` normais do Python com os sensores:

```
while True:                      # repete sem parar
    if luz_refletida(1) < 50:    # se o sensor 1 vê escuro...
        motores(10, 30)
    else:                        # ...senão
        motores(30, 10)

while distancia(4) > 15:         # anda enquanto estiver longe
    motores(30, 30)
parar()

if luz_refletida(1) < 50 and luz_refletida(2) < 50:   # os dois no preto: cruzamento
    andar_cm(5)

for _ in range(4):               # repete 4 vezes
    andar_cm(30)
    virar_direita(90)
```

Outros: `esperar_toque(4)`, `esperar_distancia(4, '<', 20)`, `esperar_luz(1, '<', 30)`, `esperar_cor(1, VERDE)`, `se_tocado(4)`, `se_luz(1, '<', 30)`, `repetir_ate_cor(1, PRETO)`.

Tela, som, luz e variáveis

Comando	O que faz
`mostrar('Oi')` / `mostrar(distancia(4), linha=2)`	escreve na tela (linhas 1, 2 e 3; a linha 1 apaga a tela)
`bip()` / `tocar(440, 0.5)` / `tocar('la', 0.5)`	som (frequência ou nota, segundos)
`melodia('do re mi fa sol')`	várias notas seguidas (`-` é pausa)
`painel(erro=..., vel=...)`	mostra até 3 valores na tela, com o nome na frente
`luz('azul')` / `apagar_luz()`	luz do robô (`'azul'`, `'roxo'`, `'vermelho'`)
`guardar('pontos', 0)` / `ler('pontos')`	guarda e lê um número

Para competição

Comando	O que faz
`calibrar_na_largada(1, 2)` e `limite_medido(1)`	mede o preto e o branco na própria pista antes de sair
`with ajustes(kp=(0.4, 0.05), vel=(25, 5)):`	muda números pelos botões do robô, sem cabo (use `ler('kp')`)
`with medir_laco(5):`	mostra quantas voltas por segundo o laço faz

Atalhos no VS Code

Digite o nome e aperte **Tab**: o modelo aparece pronto, é só trocar os números (Tab pula para o próximo).

Digite	Aparece
`programa`	a primeira linha, `from robo import *`
`para_sempre`, `repetir`, `enquanto`, `se_senao`, `funcao`	o modelo do comando
`quadrado`, `desviar`, `parar_na_linha`	um movimento pronto
`seguidor_1_sensor`, `seguidor_2_sensores`, `seguidor_proporcional`	um seguidor de linha para ajustar
`mostrar_sensor`, `painel`, `contar`, `ao_mesmo_tempo`, `esperar_botao`	tela, variáveis e outros
`calibrar_na_largada`, `ajustes`, `medir_laco`	para competição: calibrar na pista, ajustar sem cabo, medir o laço

Regras que evitam erro

- Número com casa decimal usa **ponto**: `0.5` (e não `0,5`).
- `with ...` termina com **dois-pontos** `:` e as linhas de dentro começam com **4 espaços**.
- Texto vai entre **aspas**: `mostrar('Oi')`.
- Para linha preta no chão branco, use a **luz refletida** (`luz_refletida`, `se_luz`): é mais confiável que a cor.
- Use **um modo só para cada sensor**: misturar cor e luz refletida no mesmo sensor deixa ele lento e errando (o programa avisa).
- Se der erro, leia a mensagem: ela diz **a linha** e o que corrigir.